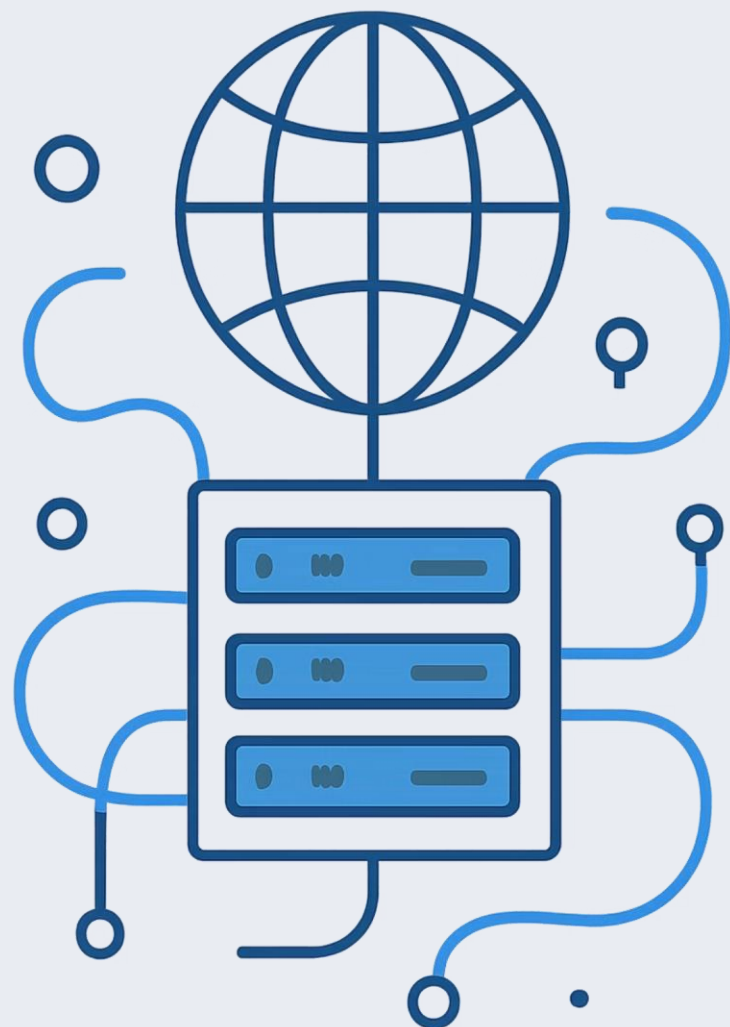




Aula 03 – Estruturação Analítica e Protocolos de Transmissão da Rede Global

Uma jornada pelo coração da web: como os dados trafegam, como os sistemas se comunicam e como arquiteturas modernas transformaram a forma como construímos aplicações. Nesta aula, dissecamos o modelo cliente-servidor, o protocolo HTTP/HTTPS e a evolução para microsserviços, culminando na prática com Flask em Python.

REDES & WEB

AULA 03

+DESAFIO TÉCNICO 02

PROF. CLOVES ROCHA

O Que Vamos Estudar Hoje

Esta aula cobre os fundamentos arquiteturais e de protocolo que sustentam toda a web moderna. Cada tópico se conecta ao próximo, formando uma visão completa do ciclo de vida de uma requisição web — da arquitetura ao código em execução.

01

1.1.1 — Modelo Cliente-Servidor

Entender a arquitetura fundamental da web, os papéis de cliente e servidor, e o ciclo completo de requisição e resposta que ocorre a cada interação.

03

1.1.3 — De Monolito a Microsserviços

Compreender a evolução histórica das arquiteturas de software, os trade-offs de cada abordagem e por que os microsserviços dominam o cenário atual.

02

1.1.2 — Protocolo HTTP/HTTPS

Dissecar a estrutura das mensagens HTTP, os cabeçalhos, os métodos, os códigos de status e a camada de segurança TLS/SSL do HTTPS.

04

Flask + Desafio Técnico 02

Apresentação do ambiente de execução Python com Flask, o miniframevork que dará vida prática aos conceitos teóricos, e detalhamento do Desafio 02.

1.1.1 – O Modelo Arquitetural Cliente-Servidor

O modelo **cliente-servidor** é a espinha dorsal de toda a web. Nele, dois atores principais se comunicam de forma assimétrica: o **cliente** inicia a comunicação solicitando recursos, e o **servidor** responde fornecendo esses recursos ou executando ações. Essa separação de responsabilidades é intencional e permite escalabilidade, segurança e manutenção independentes.

Cliente (Client)

Qualquer software que **inicia requisições**: navegadores (Chrome, Firefox), aplicativos móveis, scripts Python, ferramentas como curl ou Postman. O cliente conhece o endereço do servidor (URL), mas não precisa saber como o servidor processa internamente a solicitação.

- Formata e envia requisições HTTP
- Interpreta e exibe respostas
- Gerencia estado local (cookies, cache)

Servidor (Server)

Software que **escuta em uma porta de rede**, recebe requisições, processa a lógica de negócio e retorna respostas. Exemplos: Apache, Nginx, Flask, Django, Node.js. Um servidor pode atender milhares de clientes simultaneamente.

- Escuta em portas TCP (80, 443, etc.)
- Processa lógica e acessa bancos de dados
- Retorna respostas estruturadas (HTML, JSON)

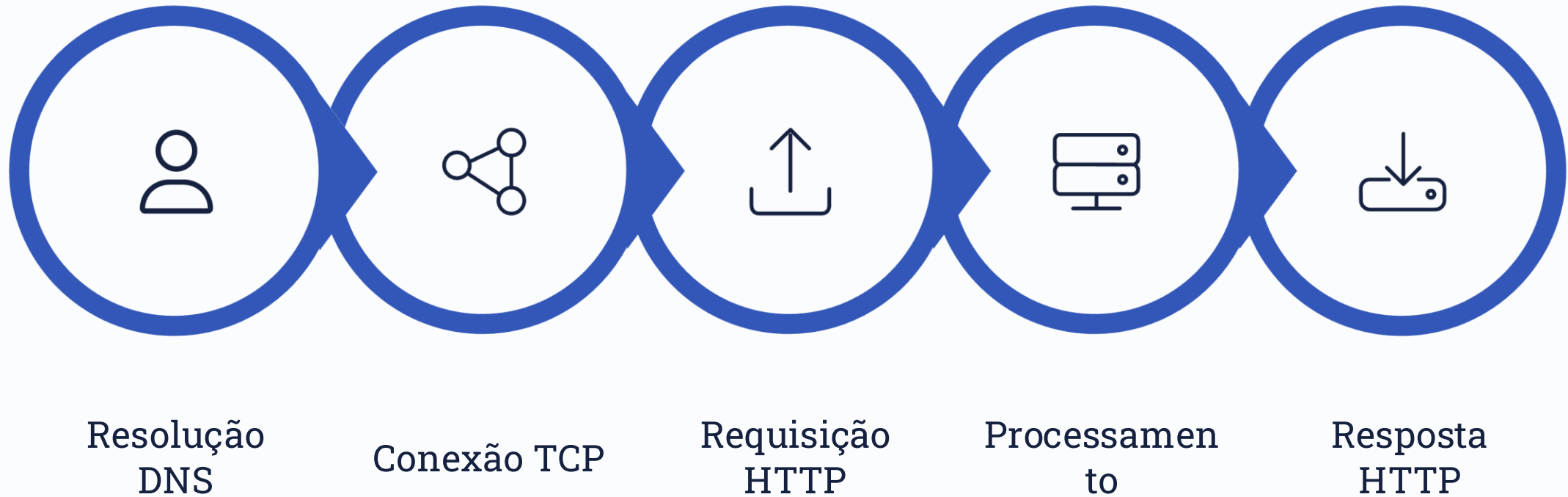
Protocolo de Comunicação

O **HTTP/HTTPS** é o contrato que define como cliente e servidor trocam mensagens. Ele especifica o formato das requisições, das respostas, os cabeçalhos, os métodos permitidos e os códigos de status que indicam o resultado de cada operação.

- Baseado em TCP/IP (camada de transporte)
- Stateless por natureza (sem estado)
- Extensível via cabeçalhos personalizados

O Ciclo de Requisição e Resposta

Cada interação na web segue um ciclo bem definido. Do momento em que você digita uma URL até a página ser renderizada, dezenas de etapas ocorrem em milissegundos. Compreender esse ciclo é essencial para diagnosticar problemas, otimizar desempenho e projetar APIs robustas.



O diagrama acima ilustra as cinco etapas fundamentais do ciclo HTTP: desde a resolução do nome de domínio até a entrega da resposta ao cliente. Cada etapa pode ser um gargalo de desempenho ou um ponto de falha que precisa ser monitorado.

Anatomia de uma Requisição HTTP

Uma requisição HTTP é composta por três partes:

- **Linha de requisição:** método + URI + versão (ex: `GET /api/users HTTP/1.1`)
- **Cabeçalhos (Headers):** metadados como `Host`, `User-Agent`, `Authorization`
- **Corpo (Body):** dados enviados (principalmente em `POST/PUT`), como JSON ou formulários

Anatomia de uma Resposta HTTP

A resposta do servidor também possui três partes:

- **Linha de status:** versão + código + descrição (ex: `HTTP/1.1 200 OK`)
- **Cabeçalhos (Headers):** metadados como `Content-Type`, `Content-Length`, `Set-Cookie`
- **Corpo (Body):** o conteúdo real — HTML, JSON, imagem, etc.

1.1.2 – Dissecando o Protocolo HTTP/HTTPS

O **HTTP (HyperText Transfer Protocol)** é o protocolo de aplicação que define como mensagens são formatadas e transmitidas na web. O **HTTPS** é a versão segura, que adiciona uma camada de criptografia TLS/SSL sobre o HTTP. A versão mais utilizada atualmente é o **HTTP/1.1**, embora o **HTTP/2** e o **HTTP/3** já estejam em ampla adoção.



Métodos HTTP (Verbos)

Definem a **ação** que o cliente deseja realizar sobre o recurso:

- **GET** — Recupera dados (idempotente, seguro)
- **POST** — Cria um novo recurso
- **PUT** — Atualiza um recurso existente (completo)
- **PATCH** — Atualização parcial
- **DELETE** — Remove um recurso
- **OPTIONS** — Consulta capacidades do servidor



Cabeçalhos (Headers) Essenciais

Metadados que acompanham toda requisição e resposta:

- **Host** — Domínio do servidor (obrigatório em HTTP/1.1)
- **Content-Type** — Formato dos dados (application/json, text/html)
- **Authorization** — Token de autenticação (ex: Bearer xyz)
- **User-Agent** — Identificação do cliente
- **Accept** — Formatos que o cliente aceita receber
- **Cache-Control** — Diretivas de cache



HTTPS e a Camada TLS

O HTTPS adiciona criptografia, autenticação e integridade:

- **Criptografia**: os dados trafegam criptografados, impedindo espionagem
- **Autenticação**: certificados digitais (X.509) validam a identidade do servidor
- **Integridade**: hashes garantem que os dados não foram alterados em trânsito
- Porta padrão: **443** (vs. 80 do HTTP)

Códigos de Status HTTP – O Vocabulário das Respostas

Os **códigos de status HTTP** são números de três dígitos que indicam o resultado de uma requisição. Eles são agrupados em cinco classes, cada uma com um significado específico. Conhecer esses códigos é fundamental para depurar APIs, entender erros e projetar sistemas resilientes.



1xx – Informativo

A requisição foi recebida e o processo continua. Raramente visto por clientes comuns.

- 100 Continue – Prosseguir com o envio
- 101 Switching Protocols – Troca de protocolo (ex: WebSocket)



2xx – Sucesso

A requisição foi recebida, compreendida e processada com sucesso.

- 200 OK – Sucesso genérico
- 201 Created – Recurso criado com sucesso
- 204 No Content – Sucesso, sem corpo na resposta



3xx – Redirecionamento

Ações adicionais são necessárias para completar a requisição.

- 301 Moved Permanently – Redirecionamento permanente
- 302 Found – Redirecionamento temporário
- 304 Not Modified – Usar versão em cache



4xx – Erro do Cliente

O cliente cometeu um erro – a requisição está mal formada ou não autorizada.

- 400 Bad Request – Sintaxe inválida
- 401 Unauthorized – Autenticação necessária
- 403 Forbidden – Acesso negado
- 404 Not Found – Recurso não encontrado



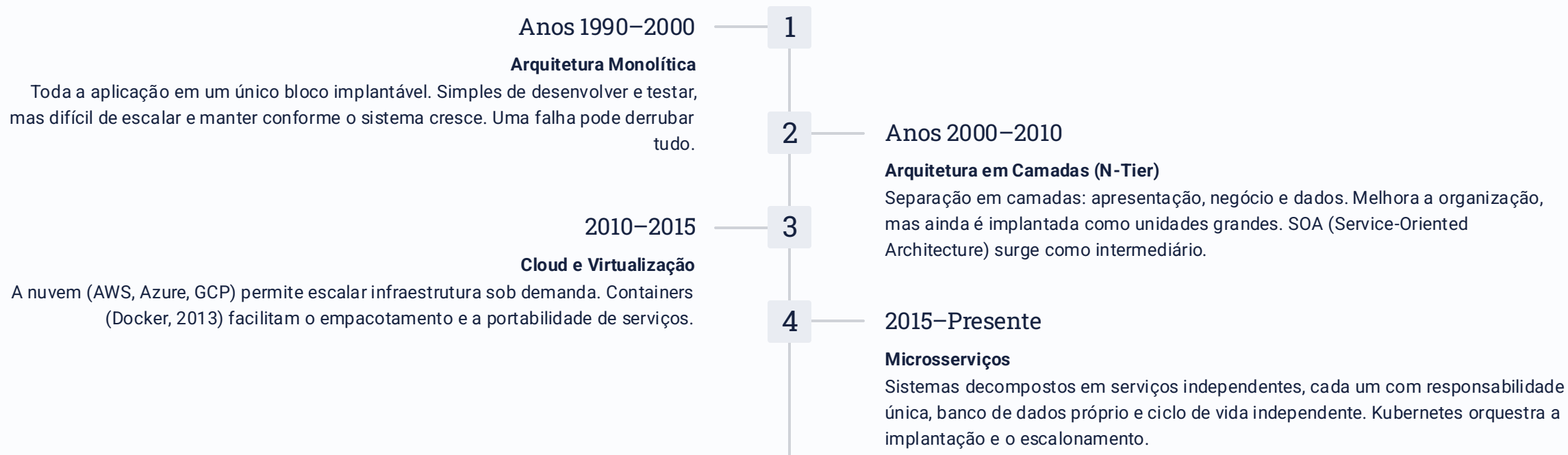
5xx – Erro do Servidor

O servidor falhou ao processar uma requisição válida.

- 500 Internal Server Error – Erro genérico do servidor
- 502 Bad Gateway – Resposta inválida do upstream
- 503 Service Unavailable – Serviço temporariamente indisponível

1.1.3 – A Evolução das Arquiteturas: Do Monolito aos Microserviços

A forma como organizamos sistemas de software evoluiu drasticamente nas últimas décadas. Cada paradigma arquitetural surgiu como resposta às limitações do anterior, impulsionado pelo crescimento da escala, da complexidade e das demandas de negócio. Entender essa evolução é crucial para tomar decisões arquiteturais informadas.



✓ Vantagens dos Microserviços

- Escalabilidade independente por serviço
- Implantações isoladas e frequentes (CI/CD)
- Falhas contidas (resiliência)
- Times autônomos por domínio
- Poli-glota: cada serviço na tecnologia ideal

⚠ Desafios dos Microserviços

- Complexidade operacional elevada
- Comunicação entre serviços (latência, falhas)
- Consistência de dados distribuídos
- Monitoramento e rastreamento distribuído
- Custo de infraestrutura maior

Flask Python – O Miniframework que Dá Vida à Teoria

Flask é um microframework web para Python, criado por Armin Ronacher em 2010. Ele é chamado de "micro" porque mantém o núcleo simples e extensível – sem ORM embutido, sem validação de formulários, sem autenticação. Você escolhe as extensões que precisa. É a ferramenta ideal para aprender HTTP na prática e construir APIs RESTful com clareza.

Por que Flask para Aprender?

Flask expõe os conceitos HTTP de forma transparente. Ao criar uma rota, você está definindo explicitamente qual método HTTP ela aceita, quais cabeçalhos retorna e qual código de status enviar. Não há magia oculta – o que você vê é o que acontece.

- Curva de aprendizado suave
- Documentação excelente e comunidade ativa
- Extensível via Flask-RESTful, Flask-SQLAlchemy, etc.
- Amplamente usado em produção (Pinterest, LinkedIn usaram Flask)

Estrutura de um Projeto Flask

Um projeto Flask típico organiza-se em módulos claros:

- `app.py` – ponto de entrada, cria a instância `Flask(__name__)`
- `routes/` – definição das rotas e handlers HTTP
- `models/` – modelos de dados (opcional, com SQLAlchemy)
- `templates/` – arquivos HTML (Jinja2)
- `static/` – CSS, JS e imagens
- `requirements.txt` – dependências do projeto

Configurando o Ambiente

Boas práticas de ambiente isolado são essenciais:

- Crie um **ambiente virtual**: `python -m venv venv`
- Ative-o: `source venv/bin/activate` (Linux/Mac) ou `venv\Scripts\activate` (Windows)
- Instale o Flask: `pip install flask`
- Execute com: `flask run` ou `python app.py`
- Servidor de desenvolvimento roda em `localhost:5000`

Passo a Passo — Criando sua Primeira API com Flask

Agora vamos transformar a teoria em código. Este exemplo mostra como criar uma API RESTful simples com Flask, cobrindo os principais métodos HTTP, códigos de status e retorno de dados em JSON. Siga cada passo com atenção.

1 Instalar e Estruturar o Projeto

Crie a pasta do projeto, o ambiente virtual e instale o Flask. Crie o arquivo `app.py` na raiz.

```
mkdir aula03-flask
cd aula03-flask
python -m venv venv
source venv/bin/activate      # Linux/Mac
venv\Scripts\activate.bat    # Windows
pip install flask
pip install flask-cors        # Para permitir requisições cruzadas
```

2 Criar a Aplicação e Definir Rotas

Escreva o código inicial com rotas para diferentes métodos HTTP. Note como cada decorator `@app.route` mapeia um método para uma função Python.

```
from flask import Flask, jsonify, request
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # Habilita CORS para todas as rotas

# Lista em memória (simulando banco de dados)
usuarios = [
    {"id": 1, "nome": "Alice", "email": "alice@exemplo.com"},
    {"id": 2, "nome": "Bob", "email": "bob@exemplo.com"}
]

@app.route('/api/usuarios', methods=['GET'])
def listar_usuarios():
    return jsonify(usuarios), 200

@app.route('/api/usuarios/<int:id>', methods=['GET'])
def obter_usuario(id):
    usuario = next((u for u in usuarios if u['id'] == id), None)
    if usuario:
        return jsonify(usuario), 200
    return jsonify({"erro": "Usuario nao encontrado"}), 404

@app.route('/api/usuarios', methods=['POST'])
def criar_usuario():
    dados = request.get_json()
    novo_id = max(u['id'] for u in usuarios) + 1
    novo_usuario = {"id": novo_id, **dados}
    usuarios.append(novo_usuario)
    return jsonify(novo_usuario), 201

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

3 Executar e Testar a API

Com o servidor rodando, use `curl` ou Postman para testar cada endpoint e observar os códigos de status e cabeçalhos retornados.

```
# Listar todos os usuarios (GET)
curl http://localhost:5000/api/usuarios

# Obter um usuario especifico (GET)
curl http://localhost:5000/api/usuarios/1

# Criar um novo usuario (POST)
curl -X POST http://localhost:5000/api/usuarios \
-H "Content-Type: application/json" \
-d '{"nome": "Carlos", "email": "carlos@exemplo.com"}'

# Tentar acessar usuario inexistente (404)
curl http://localhost:5000/api/usuarios/999
```

Desafio Técnico 02 – Detalhamento e Próximos Passos

O **Desafio Técnico 02** é a sua oportunidade de consolidar tudo que aprendemos nesta aula. Você vai construir uma API RESTful completa usando Flask, aplicando os conceitos de métodos HTTP, códigos de status, cabeçalhos e arquitetura cliente-servidor. Traga suas dúvidas para a próxima aula!

Objetivo do Desafio

Desenvolver uma API RESTful com Flask que gerencie um recurso de sua escolha (ex: produtos, tarefas, livros). A API deve implementar os cinco métodos HTTP principais e retornar os códigos de status adequados.

Requisitos Mínimos

- Rota GET `/recurso` — lista todos os itens (200)
- Rota GET `/recurso/<id>` — retorna um item (200 ou 404)
- Rota POST `/recurso` — cria um item (201)
- Rota PUT `/recurso/<id>` — atualiza um item (200 ou 404)
- Rota DELETE `/recurso/<id>` — remove um item (204 ou 404)
- Todos os retornos em `application/json`
- Tratamento de erros com mensagens descritivas

Entregáveis

- Repositório Git público (GitHub/GitLab)
- Arquivo `README.md` com instruções de instalação e uso
- Arquivo `requirements.txt` com dependências
- Código comentado e organizado
- Exemplos de requisições `curl` no README

Bônus (Opcional)

- Validação de dados de entrada (ex: email válido)
- Paginação na listagem (`?page=1&limit=10`)
- Filtro por campo (`?nome=Alice`)
- Documentação com Swagger/OpenAPI (Flask-RESTX)
- Testes unitários com `pytest`

  **Dica:** Use o Postman ou a extensão Thunder Client do VS Code para testar sua API. Documente os testes com screenshots no README para demonstrar o funcionamento.

Revisão

Revise os slides, o código da aula e a documentação oficial do Flask (flask.palletsprojects.com) antes de começar.

Dúvidas

Traga todas as dúvidas do Desafio 02 para a próxima aula. Questões sobre HTTP, Flask e arquitetura serão respondidas ao vivo.

Próxima Aula

Continuaremos com Flask aprofundado: blueprints, autenticação JWT, integração com banco de dados e deploy na nuvem.